LINKED LISTS

Prepared By-Ms. Anjli
Asst. Prof.

What is Linked List

- A linked list is a sequence of nodes where each node contains two parts:
- **Data**: The value stored in the node.
- **Pointer**: A reference to the next node in the sequence.
- Unlike arrays, linked lists do not store elements in contiguous memory locations.
- Instead, each node points to the next, forming a chain-like structure and to access any element (node), we need to first sequentially traverse all the nodes before it.

- In C, linked lists are represented as the pointer to the first node in the list.
- For that reason, the first node is generally called **head** of the linked list.
- Each node of the linked list is represented by a structure that contains a data field and a pointer of the same type as itself. Such structure is called self-referential structures.

Types of Linked List

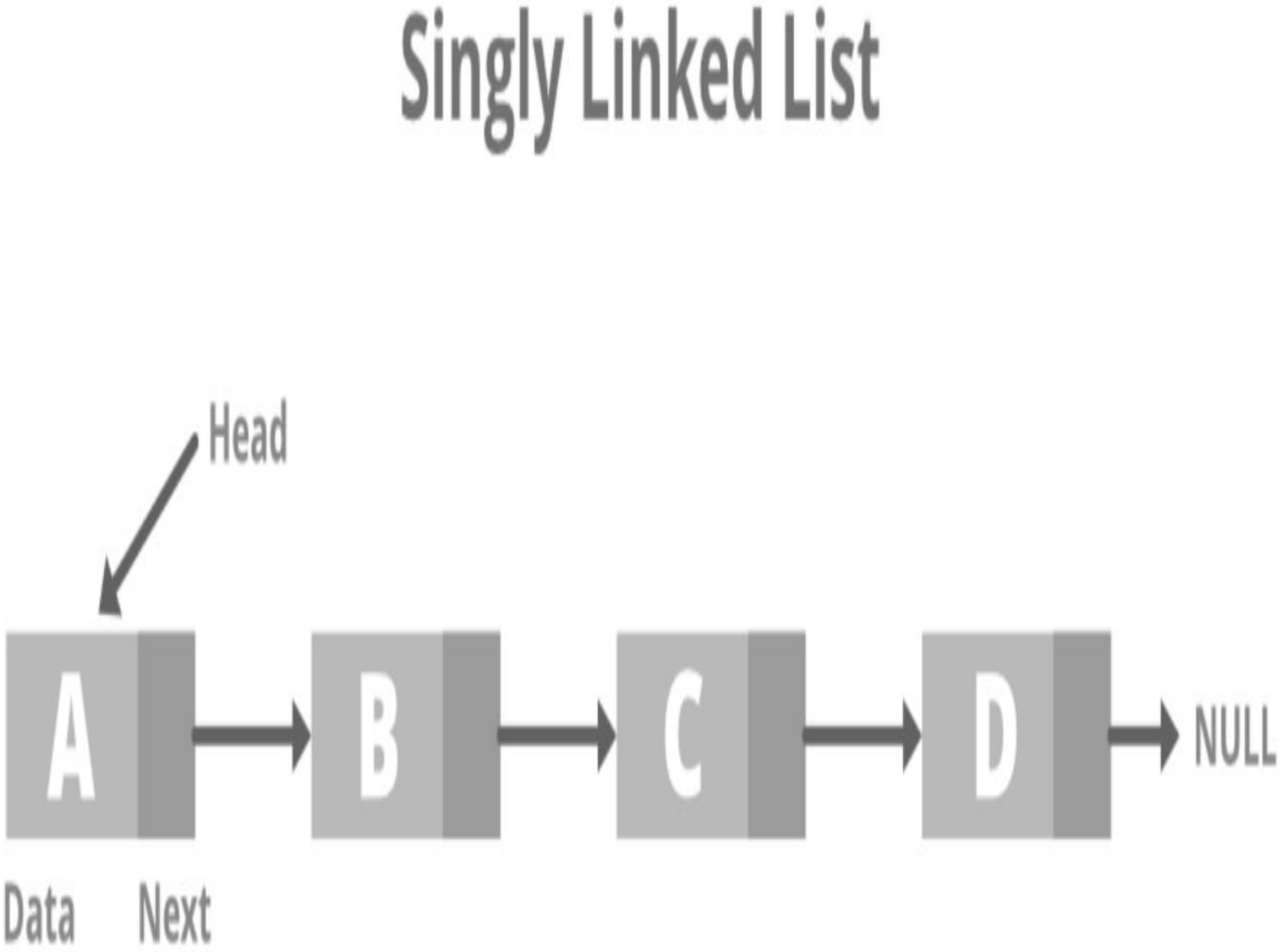
- Linked lists can be categorized based on their overall structural formation and the direction of data access. According to this classification, there are five types of linked lists:

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

Singly Linked List

- A linked list or singly linked list is a linear data structure that is made up of a group of nodes in which each node has two parts: the data, and the pointer to the next node.
- The last node's (also known as tail) pointers point to NULL to indicate the end of the linked list.

Representation



- **Data:** Here the actual information is stored.
- **Next:** Pointer that links to the next node.

Structure

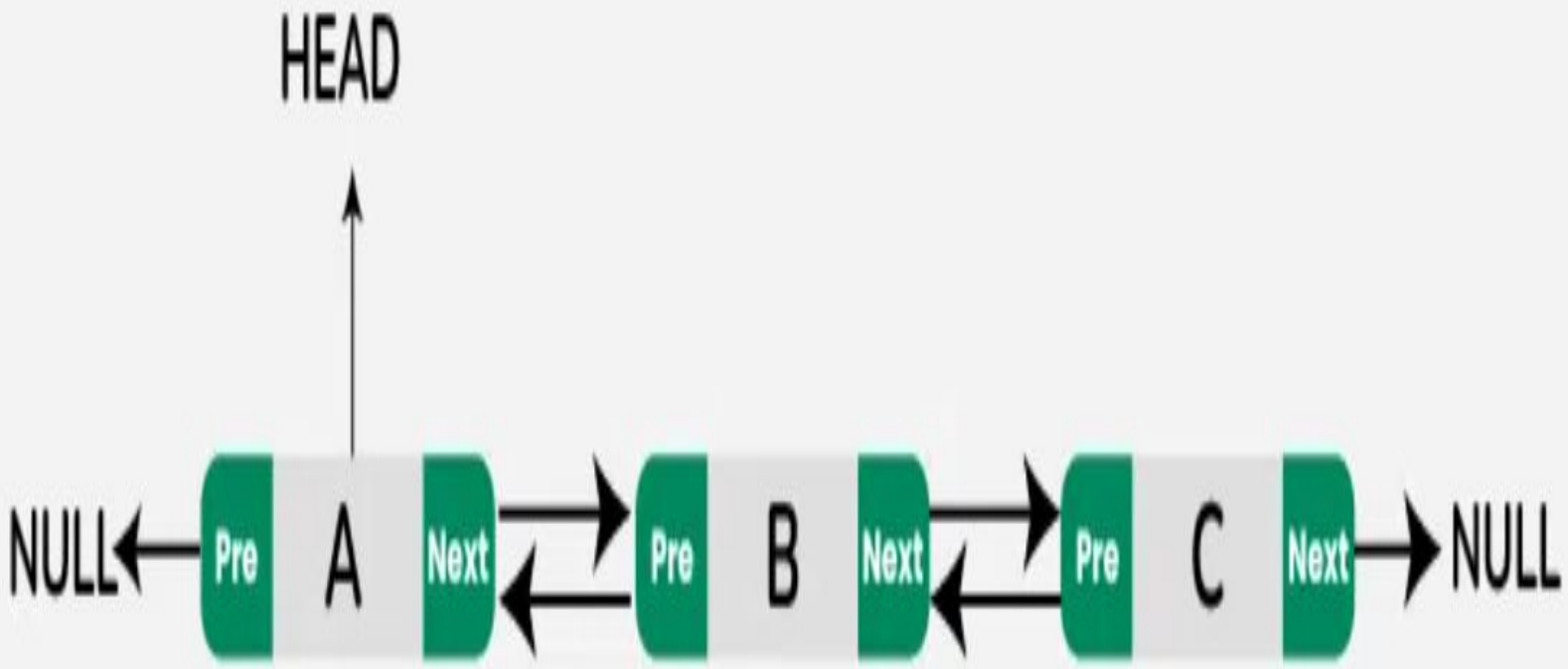
```
Struct node{
 char data;
 struct node* next;
}
```

Doubly Linked List

A **Doubly Linked List (DLL)** is a type of linked list in which each node contains three components:

1. **Data** – Stores the actual value.
2. **Pointer to the next node (next)** – Points to the next node in the list.
3. **Pointer to the previous node (prev)** – Points to the previous node in the list.

Representation



Structure

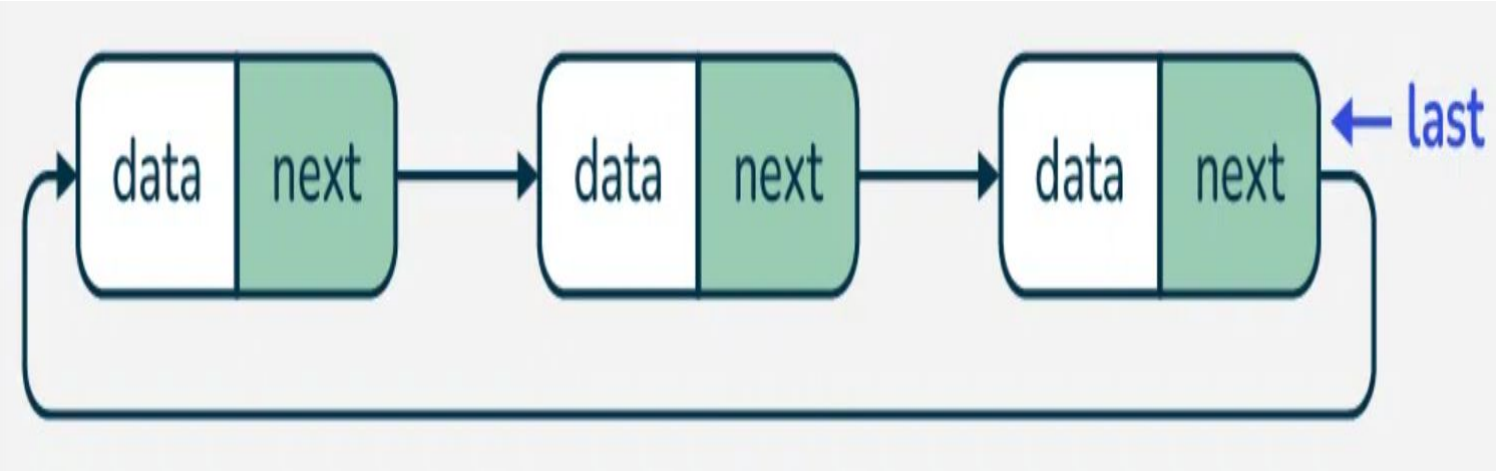
```
struct Node {
    int data;
    struct Node* next;          // Pointer to the next node
    struct Node* prev;          // Pointer to the previous node
};
```

Circular Linked List

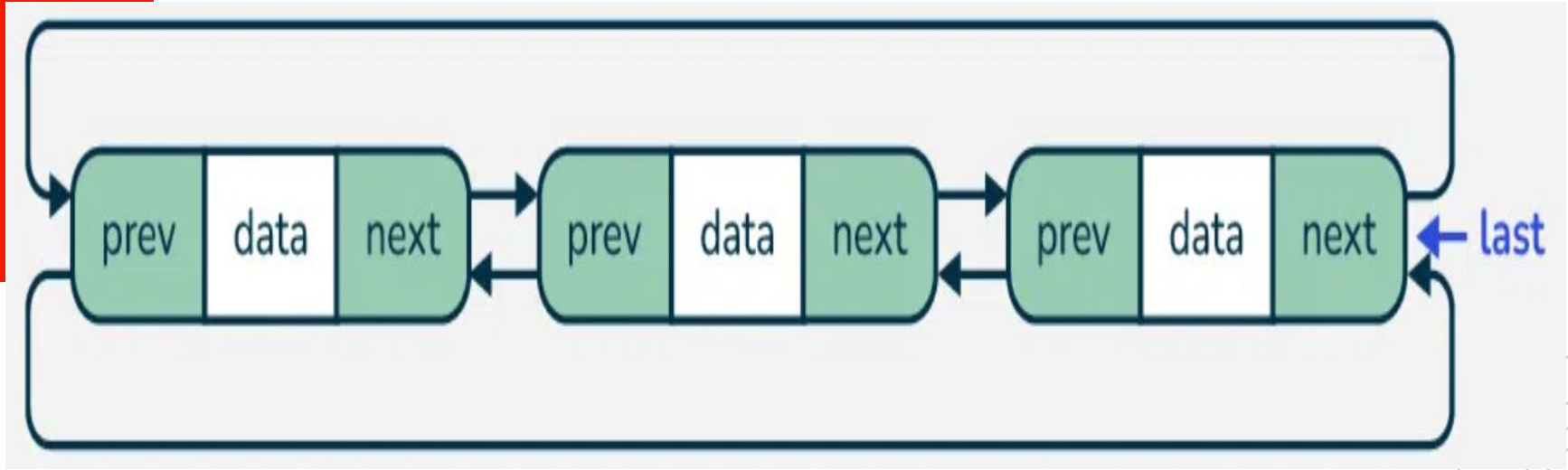
- A circular linked list is a special type of linked list where all the nodes are connected to form a circle.
- Unlike a regular linked list, which ends with a node pointing to NULL, the last node in a circular linked list points back to the first node.
- This means that you can keep traversing the list without ever reaching a NULL value.

Types of Circular Linked List

- Circular Singly List



- Circular Doubly Linked List



Operations [Singly linked list]-Insertion at Beginning

Algorithm-

Make the first node of Linked List linked to the new node

Remove the head from the original first node of Linked List

Make the new node as the Head of the Linked List.

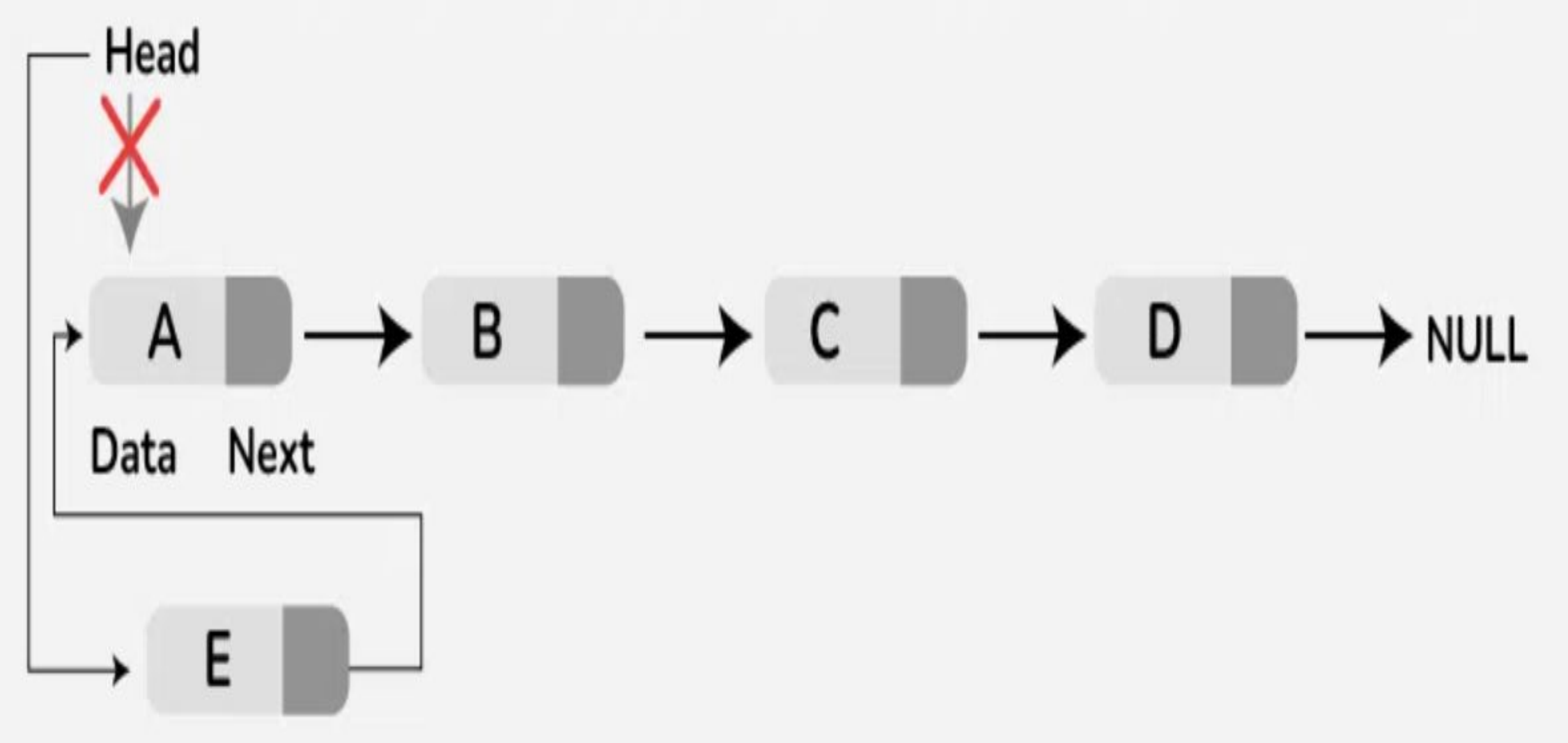
```c
#include <stdio.h>
#include <stdlib.h>

// Define a node in the linked list
struct Node {
    int data; // Data stored in the node
    struct Node* next; // Pointer to the next node in the list
};

// Function to create a new node with the given data
struct Node* createNode(int new_data)
{
    // Allocate memory for a new node
    struct Node* new_node
        = (struct Node*)malloc(sizeof(struct Node));

    // Initialize the node's data
    new_node->data = new_data;

    // Set the next pointer to NULL
    new_node->next = NULL;

    // Return the newly created node
    return new_node;
}
```

```
// Function to insert a new node at the beginning of the
// list
struct Node* insertAtFront(struct Node* head, int new_data)
{
    // Create a new node with the given data
    struct Node* new_node = createNode(new_data);

    // Make the next of the new node point to the current
    // head
    new_node->next = head;

    // Return the new node as the new head of the list
    return new_node;
}
```

```c
// Function to print the contents of the linked list
void printList(struct Node* head)
{
    // Start from the head of the list
    struct Node* curr = head;

    // Traverse the list
    while (curr != NULL) {
        // Print the current node's data
        printf(" %d", curr->data);

        // Move to the next node
        curr = curr->next;
    }

    // Print a newline at the end
    printf("\n");
}
```

```c
// Driver code to test the functions
int main()
{
    // Create the linked list 2->3->4->5
    struct Node* head = createNode(2);
    head->next = createNode(3);
    head->next->next = createNode(4);
    head->next->next->next = createNode(5);

    // Print the original list
    printf("Original Linked List:");
    printList(head);

    // Insert a new node at the front of the list
    printf("After inserting Nodes at the front:");
    int data = 1;
    head = insertAtFront(head, data);

    // Print the updated list
    printList(head);

    return 0;
}
```

```
Original Linked List: 2 3 4 5
After inserting Nodes at the front: 1 2 3 4 5



...Program finished with exit code 0
Press ENTER to exit console.
```


Head
A
B
C
D
NULL
Data
Next
E

Insertion after a given node

Algorithm

Initialize a pointer curr to traverse the list starting from head.

Loop through the list to find the node with data equal to key. If not found then return from function.

Create a new node, say new_node initialized with the given data.

Make the next pointer of new_node as next of given node.

Update the next pointer of given node point to the new_node.

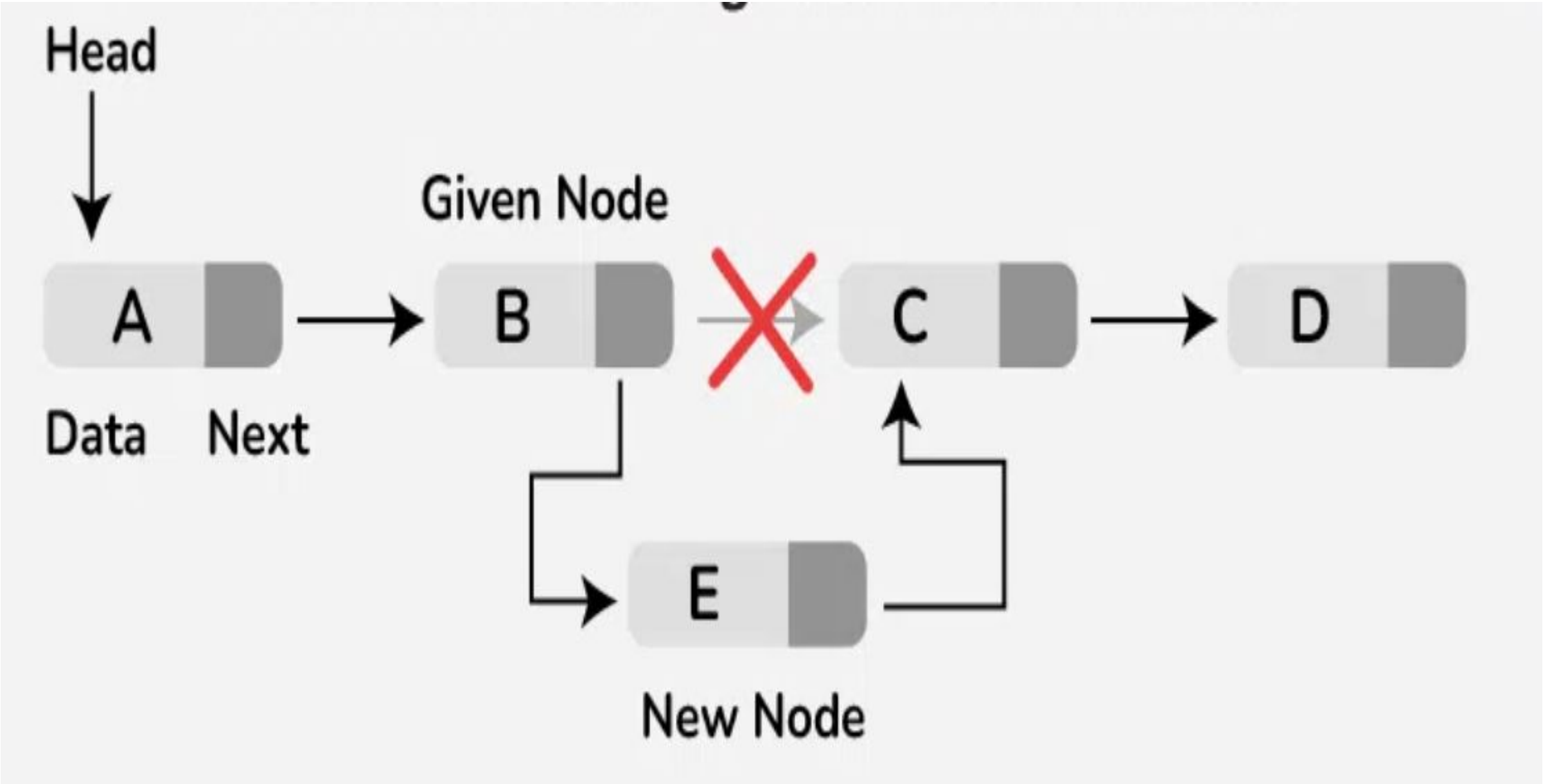
```c
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

```c
// Function to insert a node after a given node
void insertAfter(struct Node* prevNode, int newData) {
    if (prevNode == NULL) {
        printf("The given previous node cannot be NULL\n");
        return;
    }

    struct Node* newNode = createNode(newData);
    newNode->next = prevNode->next;
    prevNode->next = newNode;
}
```

```c
// Function to print the linked list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```c
int main() {
    struct Node* head = createNode(10);
    struct Node* second = createNode(20);
    struct Node* third = createNode(30);
   // Connecting nodes
    head->next = second;
    second->next = third;

    printf("Original list: ");
    printList(head);

    // Insert after second node (20)
    insertAfter(second, 25);
    printf("Updated list: ");
    printList(head);

    return 0;
}
```

```
Original list: 10 -> 20 -> 30 -> NULL
Updated list: 10 -> 20 -> 25 -> 30 -> NULL



...Program finished with exit code 0
Press ENTER to exit console.
```


Head
Given Node
A
B
C
D
Data
Next
E
New Node

Insertion at the end

Algorithm

Create a new node with the given value.

Check if the list is empty. If it is, make the new node the head and return.

Traverse the list until the last node is reached.

Link the new node to the current last node by setting the last node's next pointer to the new node.

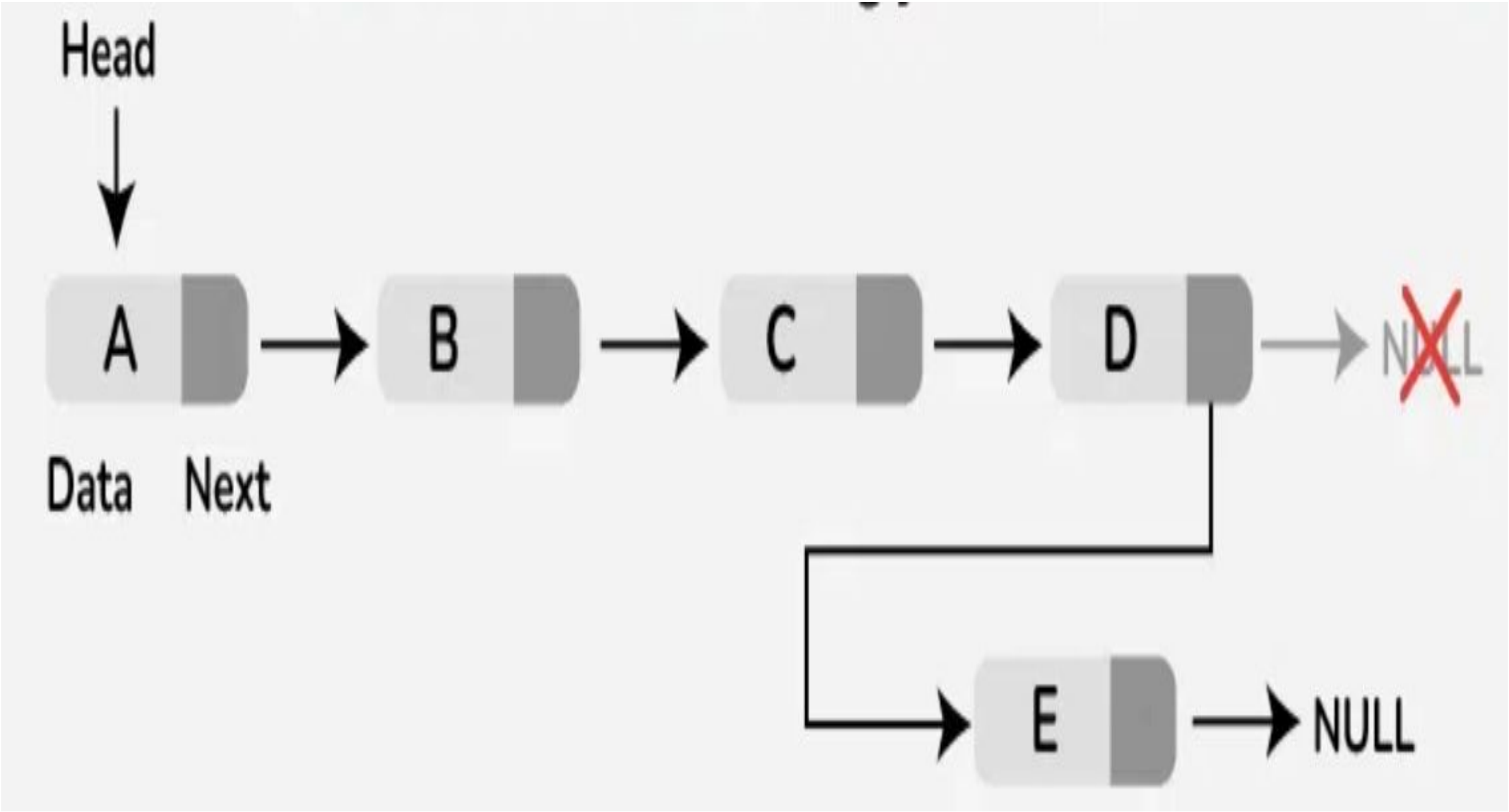
```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int new_data) {
    struct Node* new_node =
        (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = NULL;
    return new_node;
}
```

```c
struct Node* append(struct Node* head, int new_data) {

    // Create a new node
    struct Node* new_node = createNode(new_data);

    // If the Linked List is empty, make
    // the new node as the head and return
    if (head == NULL) {
        return new_node;
    }

    // Store the head reference in a temporary variable
    struct Node* last = head;

    // Traverse till the last node
    while (last->next != NULL) {
        last = last->next;
    }
```

```
    // Change the next pointer of the last node
    // to point to the new node
    last->next = new_node;

    // Return the head of the list
    return head;
}

// This function prints the contents
// of the linked list starting from the head
void printList(struct Node* node) {
    while (node != NULL) {
        printf(" %d", node->data);
        node = node->next;
    }
}
```

```c
int main() {

    struct Node* head = createNode(2);
    head->next = createNode(3);
    head->next->next = createNode(4);
    head->next->next->next = createNode(5);
    head->next->next->next->next = createNode(6);

    printf("Created Linked list is:");
    printList(head);

    // Example of appending a node at the end
    head = append(head, 1);

    printf("\nAfter inserting 1 at the end:");
    printList(head);

    return 0;
}
```

```
Created Linked list is: 2 3 4 5 6
After inserting 1 at the end: 2 3 4 5 6 1


...Program finished with exit code 0
Press ENTER to exit console.
```


Head
A
B
C
D
NULL
Data
Next
E
NULL

Deletion at the beginning of the linked list

Algorithm

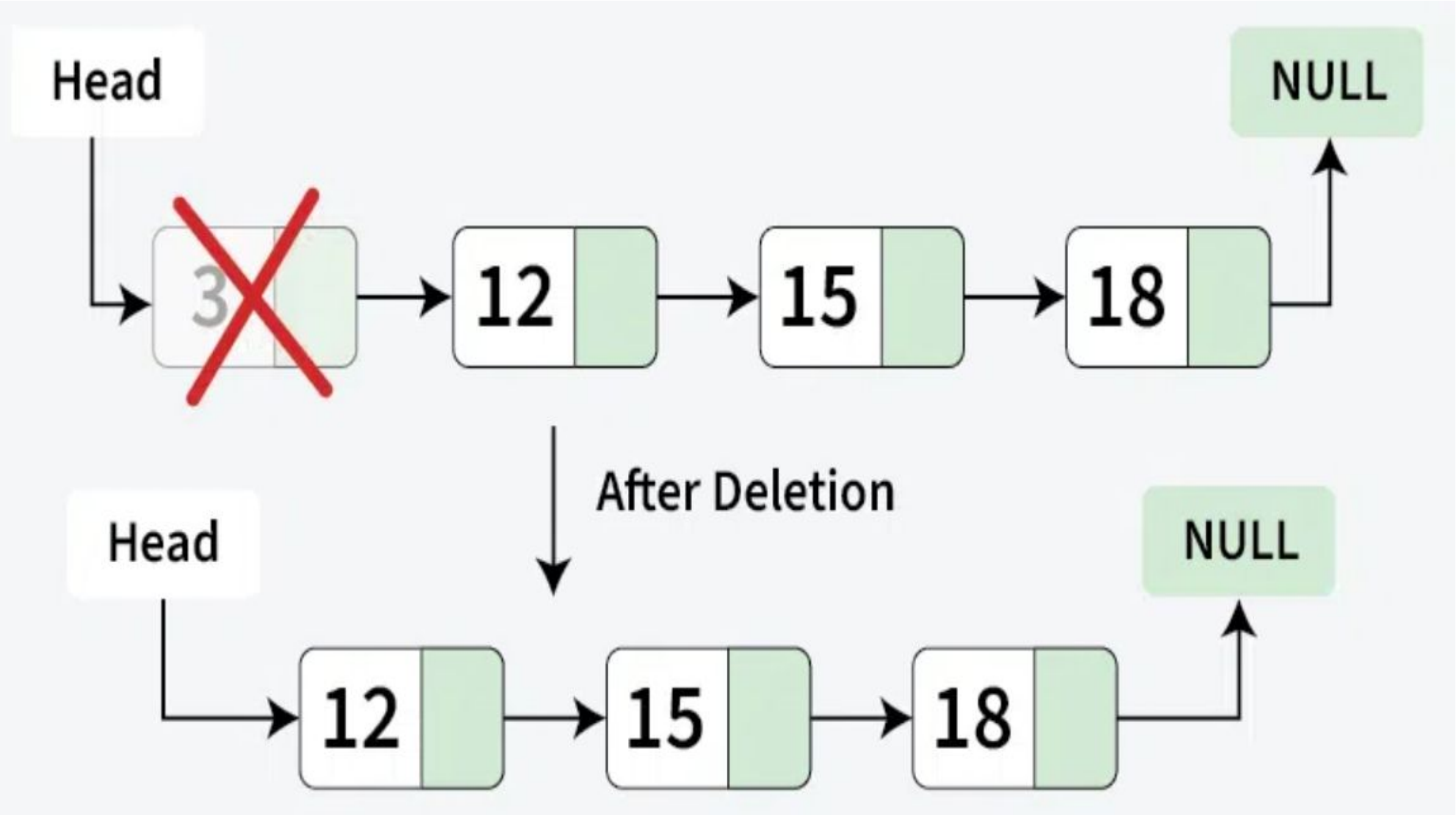
Check if the head is **NULL**. If it is, return **NULL** (the list is empty).

Store the current head node in a temporary variable **temp**.

Move the head pointer to the next node.

Delete the temporary node.

Return the new head of the linked list.


Head
NULL
3
12
15
18
After Deletion
Head
NULL
12
15
18

Deletion at the end

Algorithm

Check if the head is **NULL**.If it is, return NULL (the list is empty).

Check if the head's **next** is **NULL** (only one node in the list).If true, delete the head and return **NULL**.

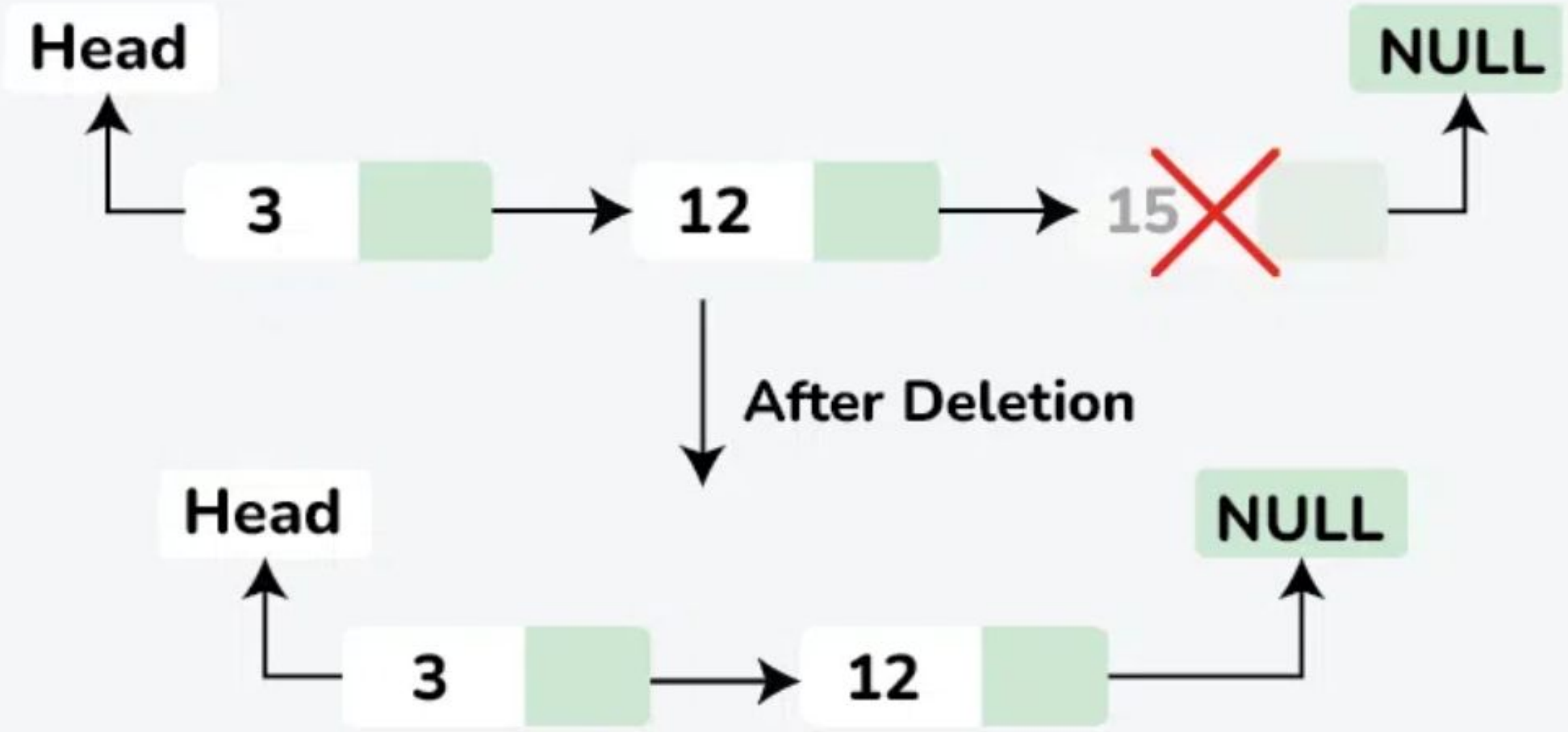
Traverse the list to find the second last node (**second_last**).

Delete the last node (the node after **second_last**).

Set the **next** pointer of the second last node to **NULL**.

Return the head of the linked list.

Deletion At End of Linked List



Deletion at specific position

- **Algorithm**

Check if the list is empty or the position is invalid, return if so.

If the head needs to be deleted, update the head and delete the node.

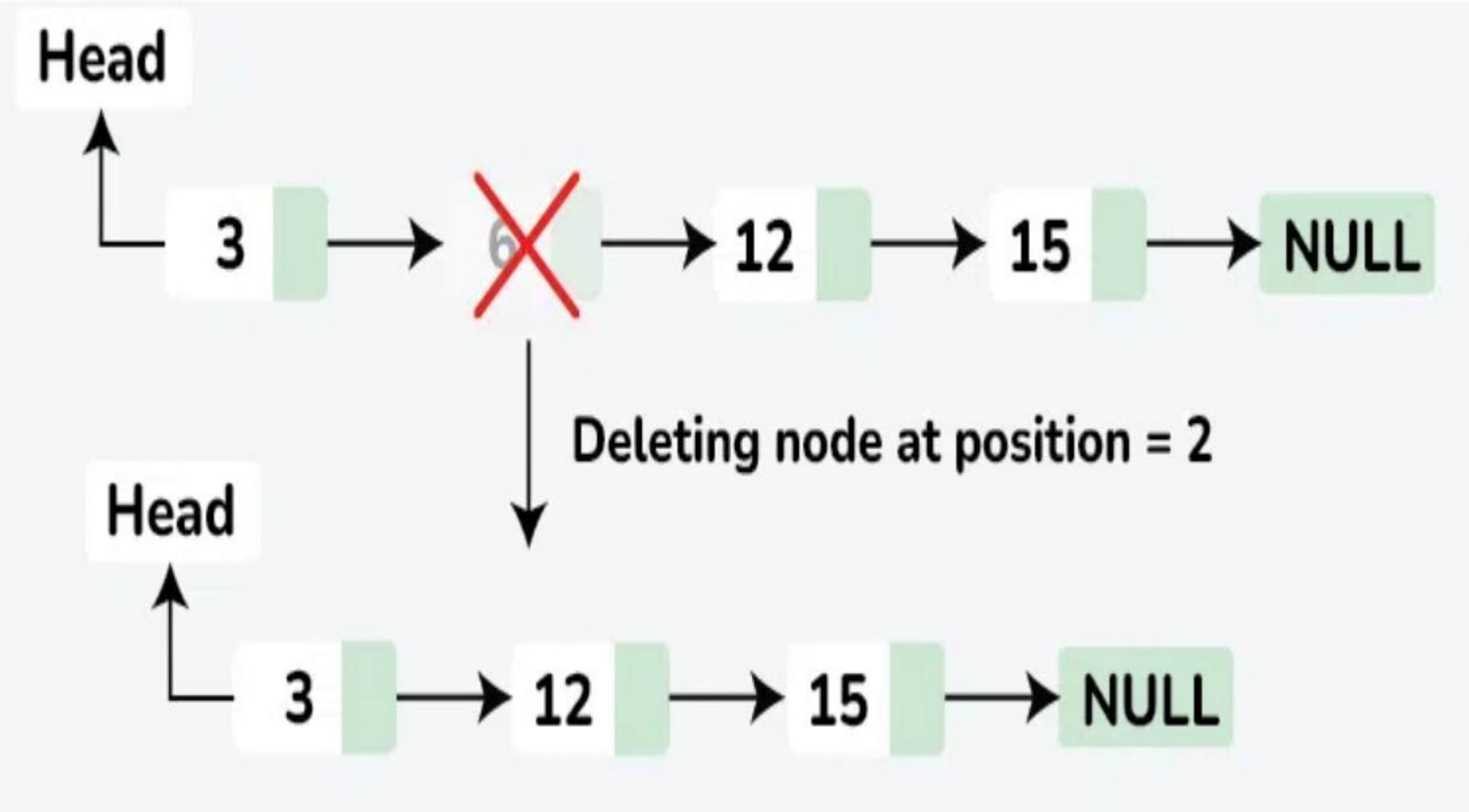
Traverse to the node before the position to be deleted.

If the position is out of range, return.

Store the node to be deleted.

Update the links to bypass the node.

Delete the stored node.


Head
3
6
12
15
NULL
Deleting node at position = 2
Head
3
12
15
NULL

Reversing SLL

- **ALGORITHM**

Initialize three pointers:

- prev (previous node, initially NULL)
- current (current node, initially pointing to the head)
- next (next node, initially NULL)
- Traverse the list:
- Store the next node (next = current->next).
- Reverse the pointer (current->next = prev).
- Move prev and current one step forward.
- Update the head pointer to the last processed node (prev).

- Void reverseList(struct node*head)

```
*prev=NULL;
*curr=head;
*next=NULL;
while(curr!=NULL)
{
next=curr->next;
curr->next=prev;
curr=next;
}
return prev;
}
```

Merging in SLL

- **ALGORITHM**
  Check if the first list (head1) is empty
- If head1 == NULL, set head1 = head2 and return.
- **Traverse to the last node of head1**
- Start from head1 and move until current->next == NULL.
- **Attach head2 to the last node of head1**
- Set current->next = head2.
- **End of algorithm**
- The concatenated list is now ready.

- Void concat(struct node *head1, struct node *head2)

```
{
struct node *ptr=Null;
ptr=head;
while(ptr->next!=Null)
{
ptr=ptr->next;
}
ptr->next=head 2;
}
```

Insertion at beginning (Doubly Linked List)

- **Algorithm**

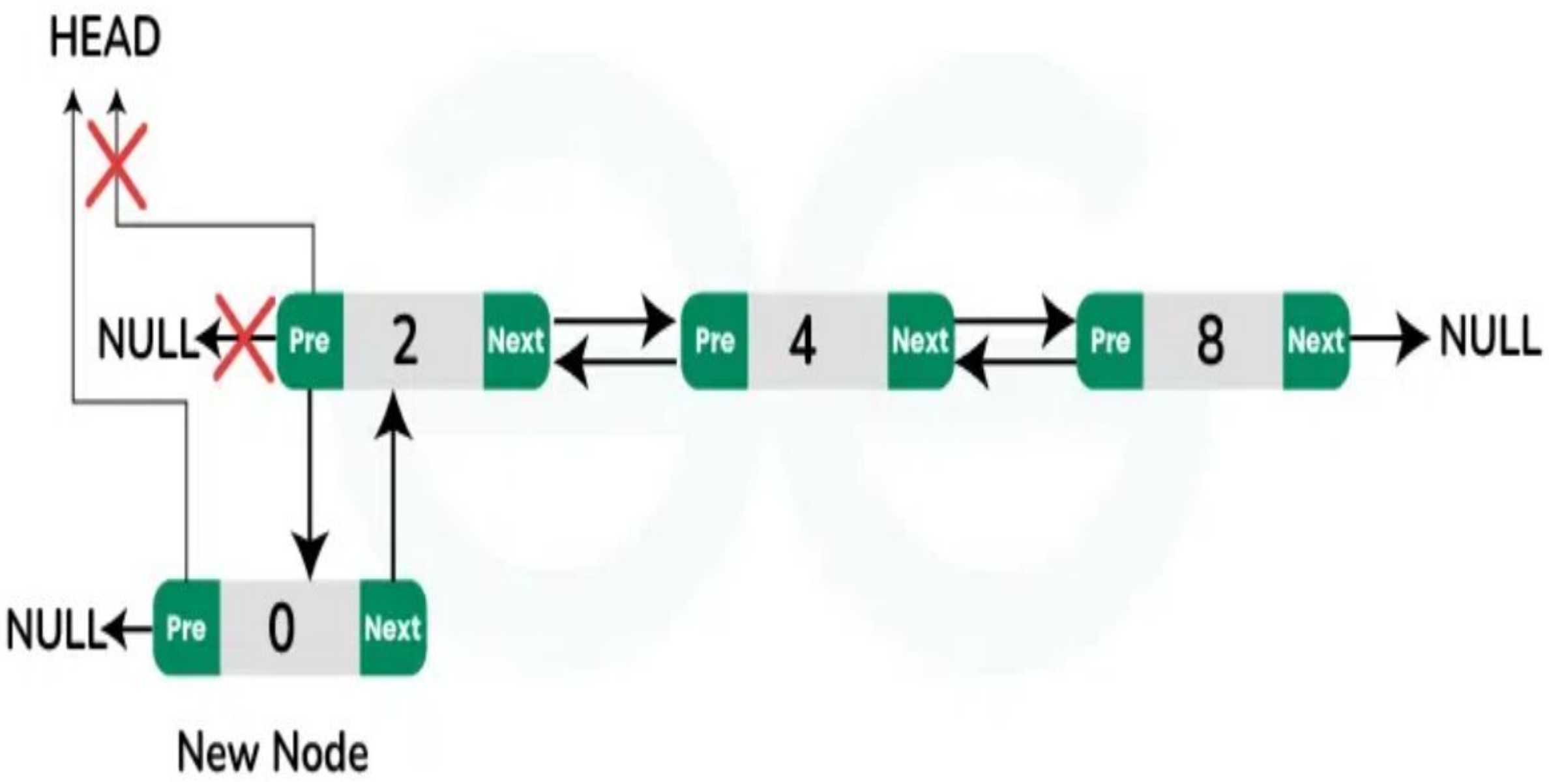
- Create a newNode with **data** field assigned the given value.
- If the list is empty:
  - Set newNode->next to NULL.
  - Set newNode->prev to NULL.
  - Update the head pointer to point to newNode.
- If the list is not empty:
  - Set newNode->next to head.
  - Set newNode->prev to NULL.
  - Set head->prev to newNode.
  - Update the head pointer to point to newNode.

```c
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

struct Node* head = NULL;

void insertAtBeginning(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;
```

```
    if (head != NULL) {
        head->prev = newNode;
    }

    head = newNode;
}

void display() {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```c
void main() {
    int choice, value;

    while (1) {
        printf("\n1. Insert at Beginning\n2. Display\n3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                insertAtBeginning(value);
                break;
            case 2:
                display();
                break;
            case 3:
                exit(0);
            default:
                printf("Invalid choice! Try again.\n");
        }
    }
}
```

```
1. Insert at Beginning
2. Display
3. Exit
Enter your choice: 1
Enter value to insert: 5

1. Insert at Beginning
2. Display
3. Exit
Enter your choice: 1
Enter value to insert: 2

1. Insert at Beginning
2. Display
3. Exit
Enter your choice: 2
Doubly Linked List: 2 <-> 5 <-> 3 <-> NULL
```


HEAD
NULL
Pre
2
Next
Pre
4
Next
Pre
8
Next
NULL
NULL
Pre
0
Next
New Node

Insertion at the end (Doubly Linked List)

- Algorithm

  Create a newNode with **data** field assigned the given value.

- If the list is empty:
  - Set newNode->next to NULL.
  - Set newNode->prev to NULL.
  - Update the head pointer to point to newNode.
- If the list is not empty:
  - Traverse the list to find the last node (where last->next == NULL).
  - Set last->next to newNode.
  - Set newNode->prev to last.
  - Set newNode->next to NULL.

```c
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

struct Node* head = NULL;

void insertAtEnd(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    newNode->data = value;
    newNode->next = NULL;
```

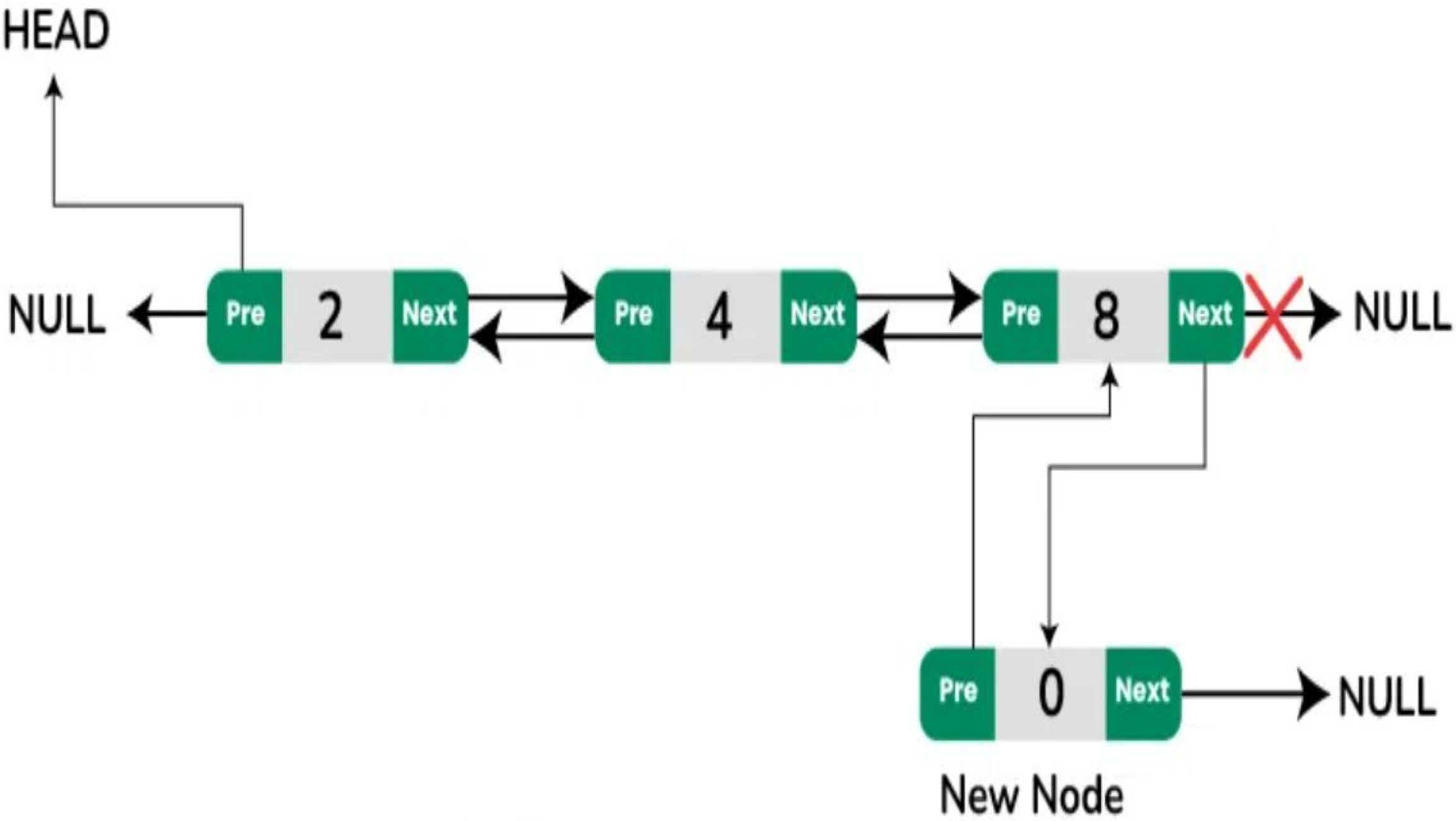
```
    if (head == NULL) {  // If list is empty, make newNode the head
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;
}

void display() {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }
```

```c
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void main() {
    int choice, value;

    while (1) {
        printf("\n1. Insert at End\n2. Display\n3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```c
        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                insertAtEnd(value);
                break;
            case 2:
                display();
                break;
            case 3:
                exit(0);
            default:
                printf("Invalid choice! Try again.\n");
        }
    }
}
```

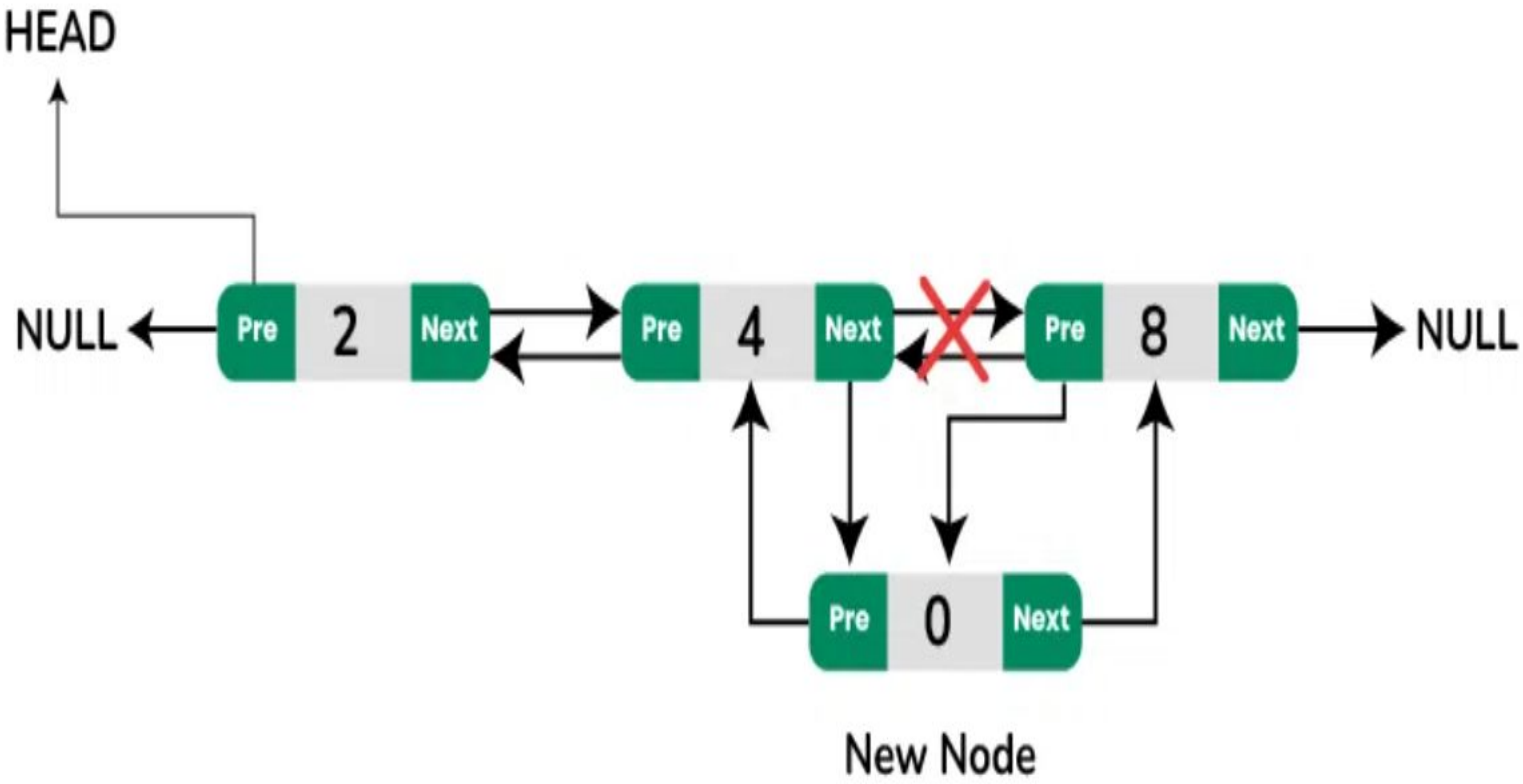
```
1. Insert at End
2. Display
3. Exit
Enter your choice: 2
List is empty

1. Insert at End
2. Display
3. Exit
Enter your choice: 1
Enter value to insert: 3

1. Insert at End
2. Display
3. Exit
Enter your choice: 1
Enter value to insert: 6
```

```
1. Insert at End
2. Display
3. Exit
Enter your choice: 1
Enter value to insert: 9

1. Insert at End
2. Display
3. Exit
Enter your choice: 2
Doubly Linked List: 3 <-> 6 <-> 9 <-> NULL
```


HEAD
NULL
Pre
2
Next
Pre
4
Next
Pre
8
Next
NULL
Pre
0
Next
NULL
New Node

Insertion in the middle of doubly linked list

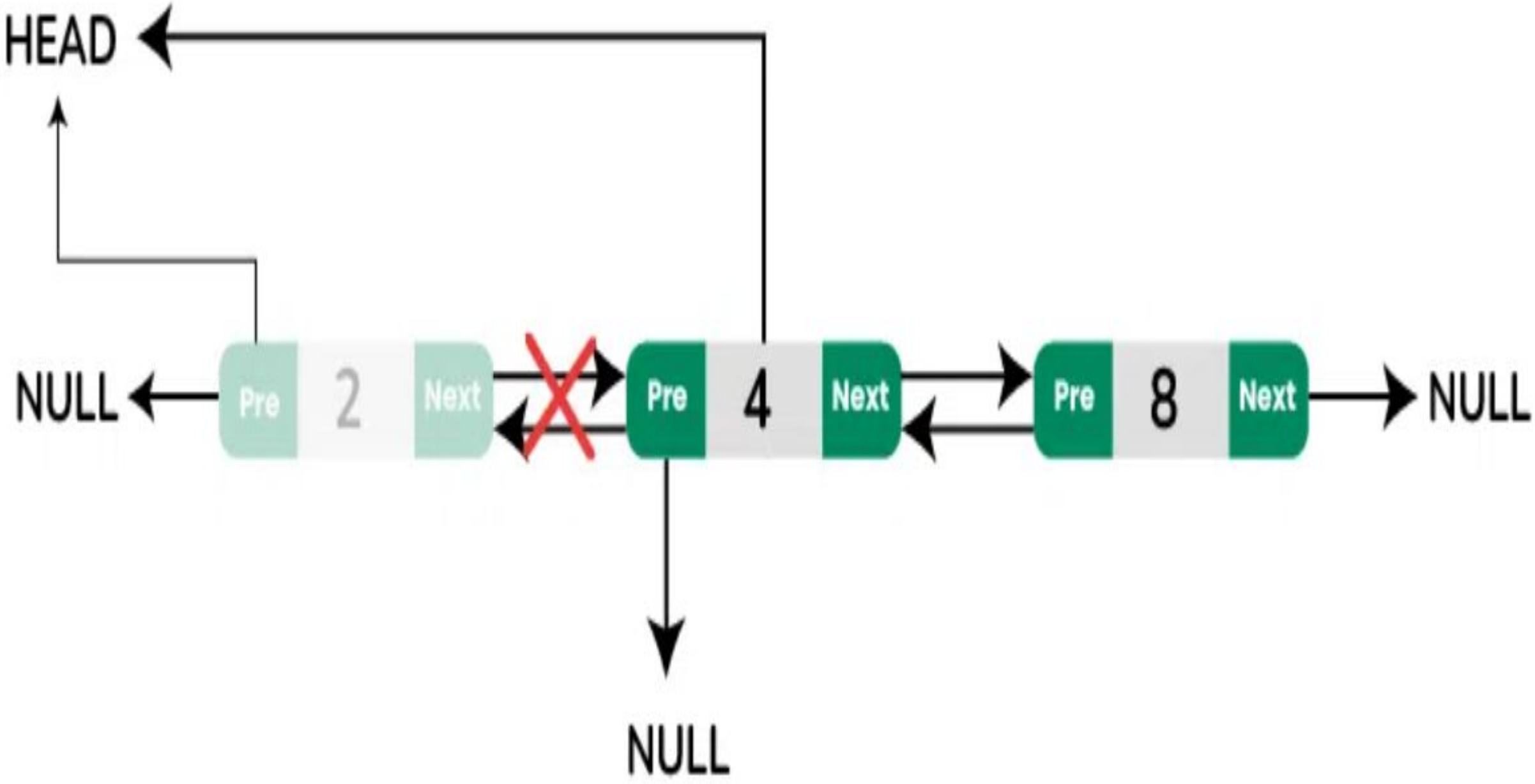
- **ALGORITHM**
- Create a newNode.
- Find the size of the list to ensure the position is within valid bounds.
- If the position is 0 (or 1 depending on your indexing):
  - Use the insertion at the beginning approach.
- If the position is equal to the size of the list:
  - Use the insertion at the end approach.
- If the position is valid and not at the boundaries:
  - Traverse the list to find the node immediately before the desired insertion point (prevNode).
  - Set newNode->next to prevNode->next.
  - Set newNode->prev to prevNode.
  - If prevNode->next is not NULL, set prevNode->next->prev to newNode.
  - Set prevNode->next to newNode.


HEAD
NULL
Pre
2
Next
Pre
4
Next
Pre
8
Next
NULL
Pre
0
Next
New Node

Deletion at the beginning

- **Algorithm**

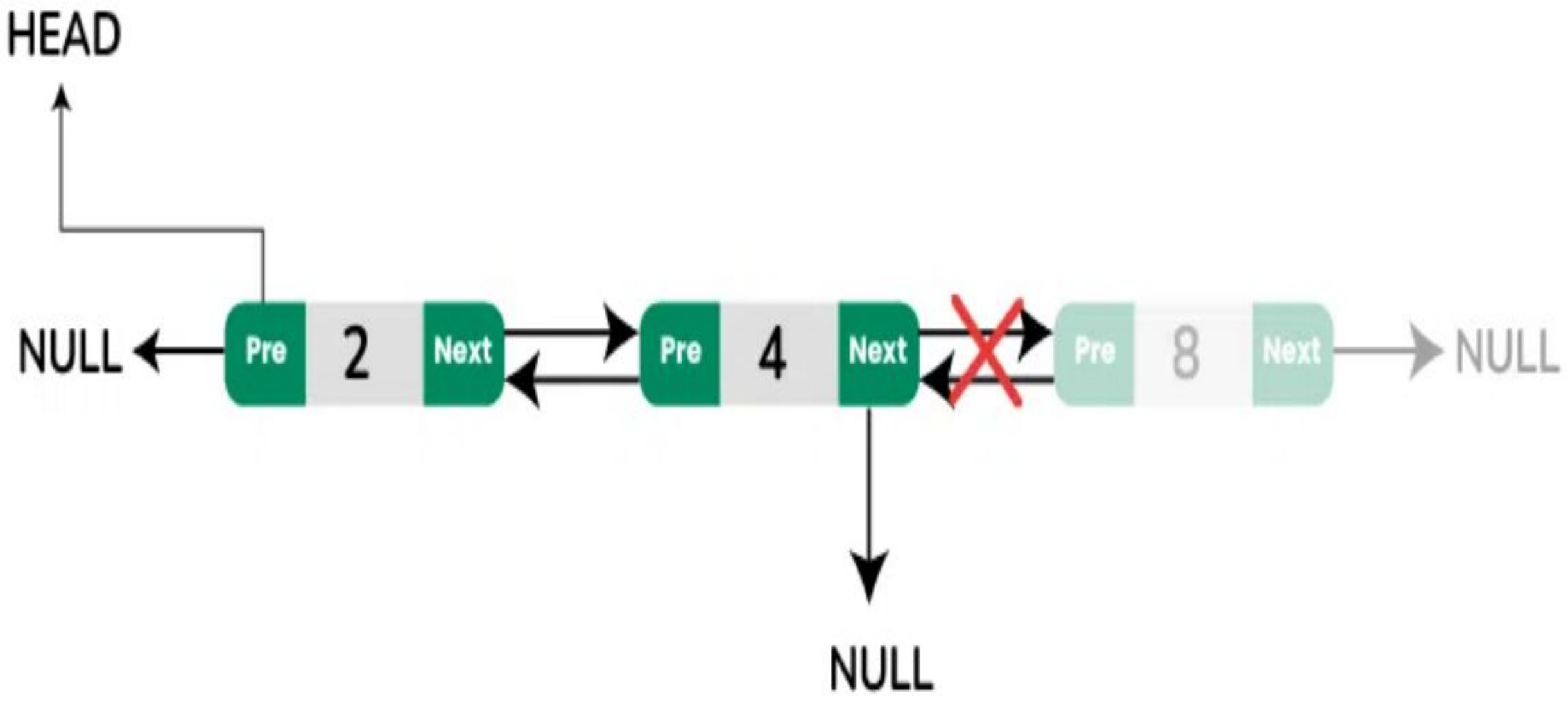
- **Check if the List is Empty**: If true, there's nothing to delete.
- **Check if the List Contains Only One Node**:
  - Set head to NULL.
  - Free the memory of the node.
- **If the List Contains More Than One Node**:
  - Update head to head->next.
  - Set the prev of the new head to NULL.
  - Free the memory of the old head.


HEAD
NULL
Pre
2
Next
Pre
4
Next
Pre
8
Next
NULL
NULL

Deletion at the end

- **Algorithm**

- Check if the List is Empty: If true, there's nothing to delete.
- If the List is Not Empty:
  - Traverse to the last node using a loop.
  - Check if the last node is the only node (head->next == NULL):
    - Update head to NULL.
  - If more than one node:
    - Set the next of the second last node (last->prev) to NULL.
  - Free the memory of the last node.


HEAD
NULL
Pre
2
Next
Pre
4
Next
Pre
8
Next
NULL
NULL

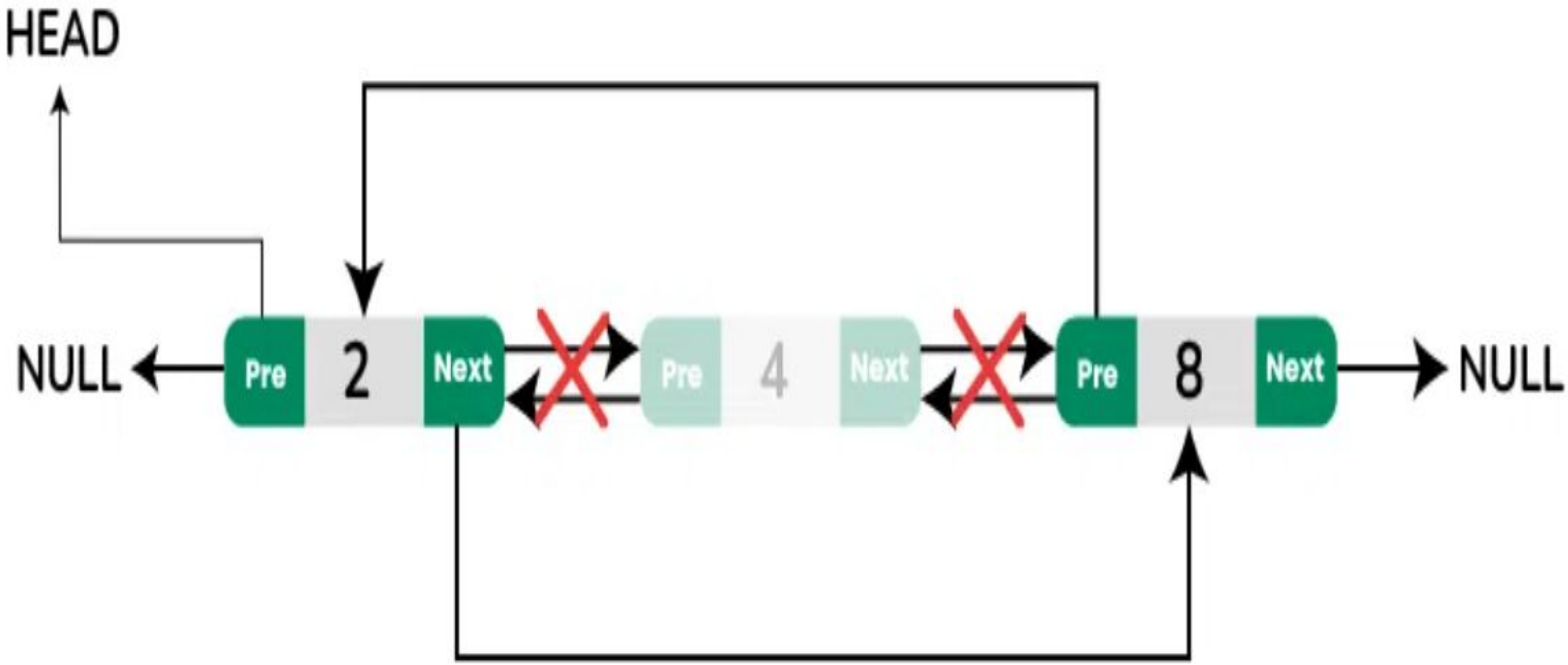
Deletion at specific position

Algorithm

Check Position Validity:

- If position is 0 (or 1 depending on indexing), use deletion at the beginning.
- If position is the size of the list, use deletion at the end.
- Otherwise, proceed with the middle deletion.

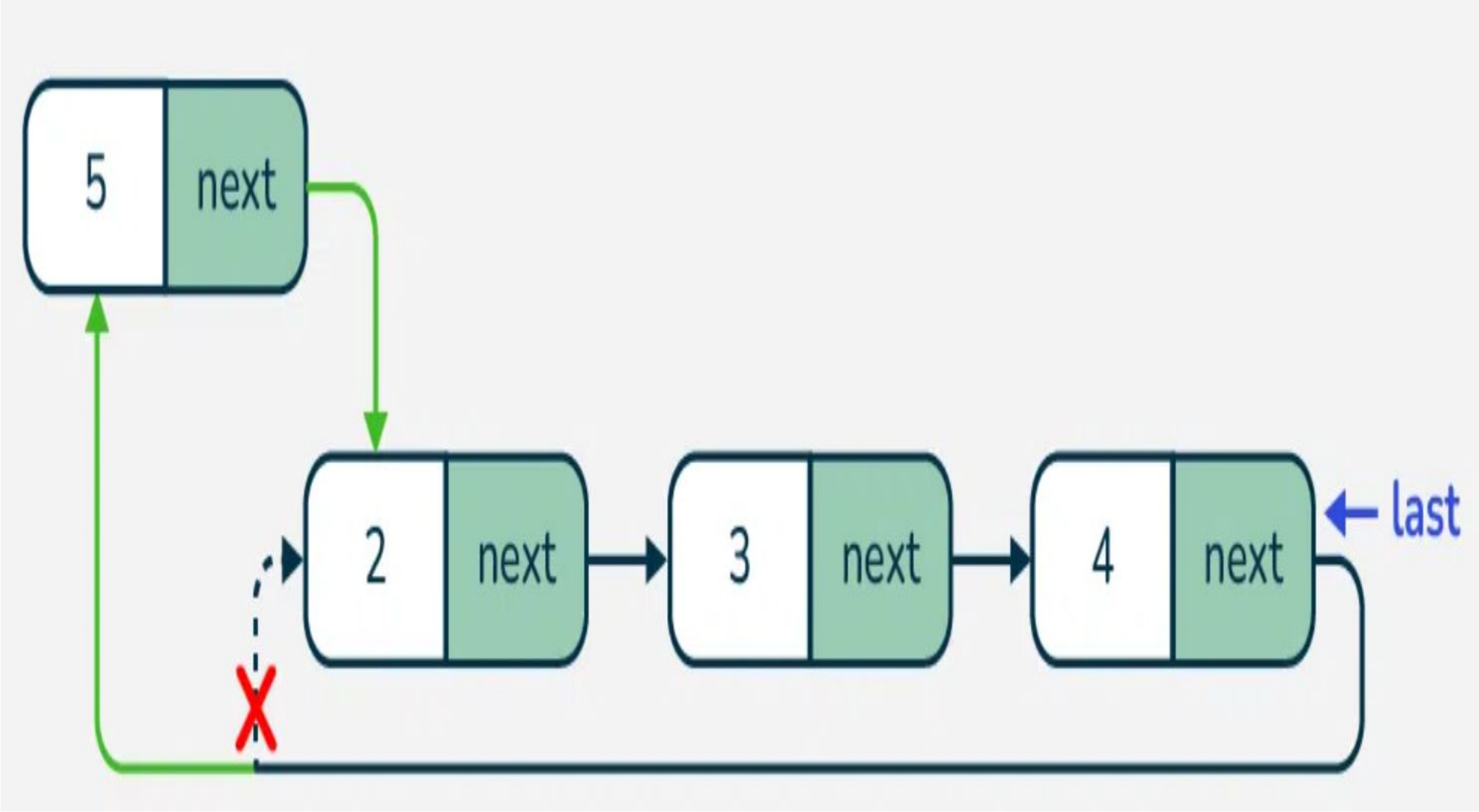
- **Traverse to the Specified Position**:
  - Use a loop to reach the node (delNode) at the desired position.
  - If delNode is the first node, adjust head.
  - If not, adjust delNode->prev->next and delNode->next->prev if delNode->next is not NULL.
- **Free the Memory**:
  - Release the node to be deleted.


HEAD
NULL
Pre
2
Next
Pre
4
Next
Pre
8
Next
NULL

Circular SLL [Insertion at the beginning]

- ALGORITHM

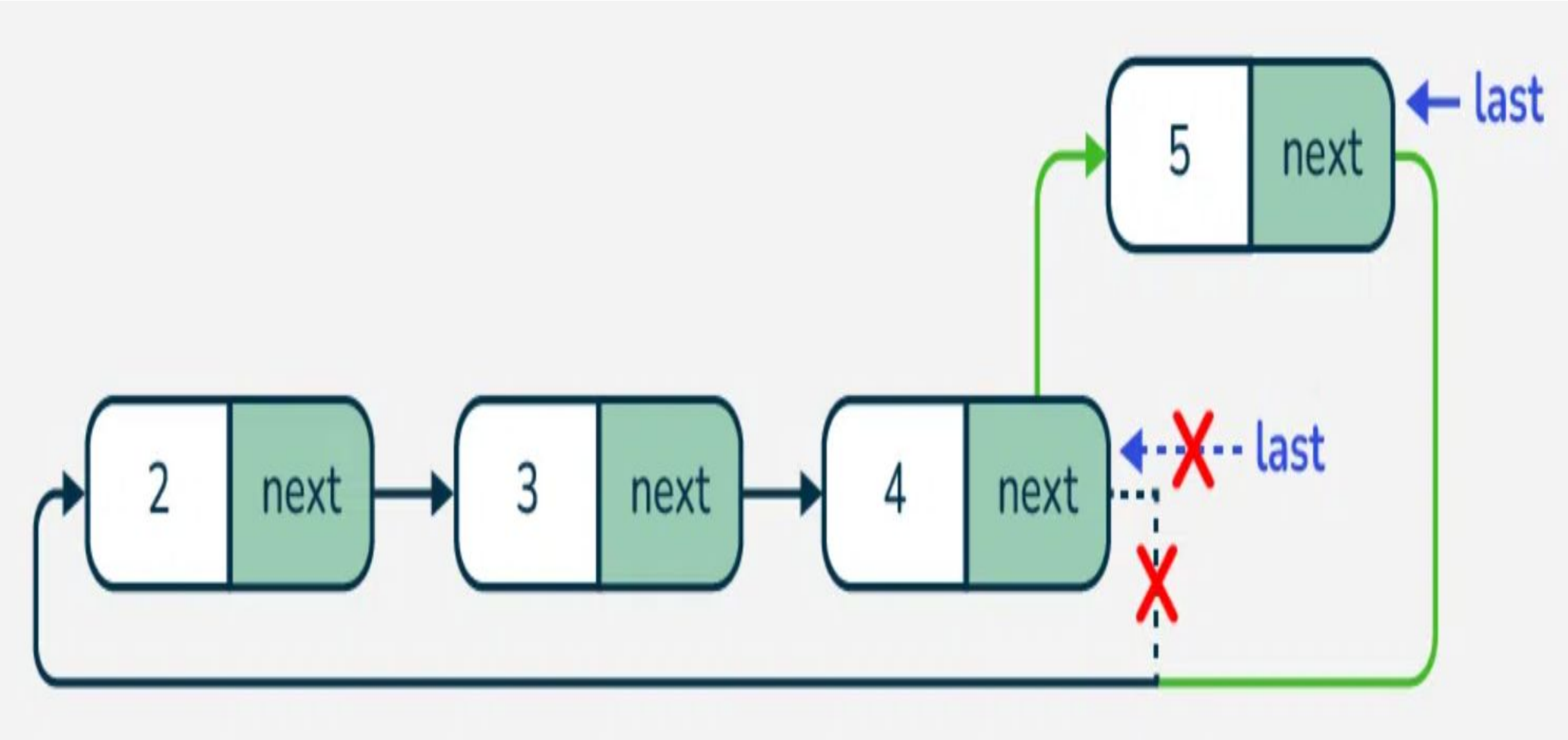
- Create a new node with the given value.
- Check Empty List (**last == nullptr**):
  - Make **newNode->next** point to itself.
- Insert at Beginning:
  - Set **newNode->next** to **last->next**.
  - Update **last->next** to **newNode**.


5
next
2
next
3
next
4
next
last

Insertion at the end

- ALGORITHM

- Create a new node with the given value.
- Check Empty List, If **last** is **nullptr** then initialize the list with the new node and make it point to itself.
- Otherwise, Set the **new node's** next to the **head** node.
- Update the current **last's** next to point to the **new node**.
- Update **last** to the **new node.**


5
next
last
2
next
3
next
4
next
last